Dependency Injection w Springu od podstaw

---

1. Czym jest zależność

Zależność to coś, czego klasa potrzebuje do działania.

W kodzie: klasa `OrderService` (obsługa zamówień) potrzebuje klasy `Sender` (wysyła potwierdzenia), więc `OrderService` zależy od `Sender`.

W prawdziwej aplikacji prawie każda klasa od czegoś zależy: controller od service, service od repository, repository od połączenia z bazą.

2. Problem: kod bez DI

Najprostsze rozwiązanie to utworzyć zależność samemu, słowem `new`:

```
public class EmailSender {
    public void send(String text) {
        System.out.println("E-mail: " + text);
    }
}

public class OrderService {

    private final EmailSender sender = new EmailSender(); // klasa sama tworzy
zależność

    public void placeOrder(String product) {
        sender.send("Zamówienie przyjęte: " + product);
    }
}
```

Działa, ale ma cztery wady:

1. **Sztywne powiązanie.** `OrderService` na zawsze zna konkretną klasę `EmailSender`.
2. **Trudne testowanie.** Nie da się podmienić `EmailSender` na atrapę, więc test wysłałby prawdziwego maila.
3. **Trudne tworzenie.** Gdy zależność sama ma zależności (np. połączenie z bazą, konfigurację), każda klasa musi wiedzieć, jak je zbudować.
4. **Marnowanie zasobów.** Każde `new` tworzy nowy obiekt. Zamiast jednego połączenia z bazą możemy mieć ich dwadzieścia.

3. Rozwiązanie: IoC i DI

To samo robimy w kodzie. Klasa mówi, czego potrzebuje, a ktoś z zewnątrz jej to dostarcza:

```
public interface Sender {
    void send(String text);
}

public class OrderService {

    private final Sender sender; // tylko deklaracja: czego potrzebuję

    public OrderService(Sender sender) { // ktoś mi to poda w konstruktorze
        this.sender = sender;
    }

    public void placeOrder(String product) {
        sender.send("Zamówienie przyjęte: " + product);
    }
}
```

Teraz `OrderService` nie wie, czy dostaje e-mail, SMS czy atrapę z testu. Wie tylko, że dostaje coś, co umie `send`.

Dwa pojęcia, które trzeba odróżnić

- **IoC (Inversion of Control, odwrócenie kontroli):** ogólna zasada. To nie klasa tworzy swoje zależności, tylko ktoś z zewnątrz. Kontrola nad tworzeniem obiektów zostaje „odwrócona".
- **DI (Dependency Injection, wstrzykiwanie zależności):** konkretny sposób realizacji tej zasady. Zależności są przekazywane obiektowi z zewnątrz (np. przez konstruktor).

DI można robić bez Springa, ręcznie: `new OrderService(new EmailSender())`. Spring robi to automatycznie.

4. Jak Spring to robi

Spring ma w środku kontener (nazywany `ApplicationContext`), czyli miejsce, w którym trzyma gotowe obiekty. Obiekty zarządzane przez Springa nazywamy beanami.

Co dzieje się przy starcie aplikacji:

1. Uruchamia się `SpringApplication.run(...)`.
2. Spring skanuje pakiet klasy z `@SpringBootApplication` oraz wszystkie jego podpakiety i szuka klas z odpowiednimi adnotacjami.
3. Dla każdej znalezionej klasy zapisuje „przepis" na bean.
4. Czyta konstruktory i buduje graf zależności: wie, że A potrzebuje B, a B potrzebuje C.
5. Tworzy obiekty od końca: najpierw te bez zależności (C), potem te, które ich potrzebują (B, potem A).
6. Wywołuje konstruktory, podając gotowe obiekty.
7. Gotowe beany leżą w kontenerze, a aplikacja działa.

```
EmailSender  →  OrderService  →  OrderController
(tworzony 1.)   (tworzony 2.)    (tworzony 3.)
```

Adnotacje, które robią z klasy bean

Adnotacja	Do czego
`@Component`	Ogólny bean
`@Service`	Logika biznesowa
`@Repository`	Dostęp do danych
`@RestController`	Obsługa żądań HTTP
`@Configuration`	Klasa z ręczną konfiguracją beanów (patrz część 8)

`@Service`, `@Repository` i `@RestController` to w gruncie rzeczy `@Component` z dodatkowym znaczeniem. Dla DI działają tak samo, ale nadają kodowi czytelną rolę (a w przypadku `@RestController` i `@Repository` dodają też własne funkcje).

Ważne: Spring widzi tylko klasy w pakiecie głównym aplikacji i jego podpakietach. Klasa w innym pakiecie, nawet z adnotacją, zostanie pominięta.

5. Trzy sposoby wstrzykiwania

```
// 1. Przez konstruktor (ZALECANE)
@Service
public class OrderService {
    private final Sender sender;

    public OrderService(Sender sender) {
        this.sender = sender;
    }
}

// 2. Przez pole (niezalecane)
@Service
public class OrderService {
    @Autowired
    private Sender sender;
}

// 3. Przez setter (rzadko)
@Service
public class OrderService {
    private Sender sender;

    @Autowired
    public void setSender(Sender sender) {
        this.sender = sender;
    }
}
```

	Konstruktor	Pole	Setter
Pole może być `final`	tak	nie	nie
Obiekt zawsze kompletny	tak	nie	nie
Łatwy test bez Springa	tak (`new OrderService(atrapa)`)	trudny	możliwy
Widać zależności	od razu w konstruktorze	ukryte w polach	rozproszone
Kiedy używać	domyślnie zawsze	unikać	zależność opcjonalna

Dlaczego konstruktor wygrywa:

- Obiekt nie może powstać „w połowie”, bez swoich zależności.
- Pole `final` gwarantuje, że nikt go później nie podmieni.
- Konstruktor z wieloma parametrami (np. ośmioma) to **sygnał ostrzegawczy**, że klasa robi za dużo. Przy wstrzykiwaniu do pól ten problem jest niewidoczny.
- Przy jednym konstruktorze adnotacja `@Autowired` **nie jest potrzebna**.

6. Do czego używać DI, a do czego nie

Prosta zasada: rzeczy, które coś ROBIĄ, to beany. Rzeczy, które SĄ danymi, tworzymy przez `new`.

Beany (wstrzykujemy)	Zwykłe obiekty (`new`)
Controller, Service, Repository	Encje, np. `new Book("Lalka", "Prus", 1890)`
Klient HTTP, wysyłacz maili	obiekty do przesyłania danych
Konfiguracja, połączenie z bazą	Wartości: `String`, `LocalDate`, liczby
Obiekty wspólne dla całej aplikacji	Obiekty tworzone w dużej liczbie, w locie

Kiedy DI daje najwięcej

- Zależność może mieć kilka wersji (e-mail czy SMS, płatność kartą czy BLIK).
- Chcesz testować z atrapami, bez prawdziwej bazy i prawdziwego wysyłania.
- Ustawienia różnią się zależnie od środowiska (testowe, produkcyjne).
- Zasób ma być wspólny (jedno połączenie z bazą).

Kiedy @Component nie wystarczy: @Configuration i @Bean

Nie możesz dodać adnotacji do klasy z biblioteki albo z Javy. Wtedy beana tworzysz ręcznie w klasie konfiguracyjnej:

```
@Configuration
public class AppConfig {

    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();
    }
}
```

Od tej chwili każda klasa może poprosić w konstruktorze o `Clock` i Spring go poda. W testach można podstawić zegar z ustaloną godziną.

9. Kompletny przykład

Projekt Spring Boot (pakiet `pl.example.Spring`). Każda klasa w osobnym pliku.

Application.java (klasa startowa)

```
package pl.example.Spring;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Sender.java

```
package pl.example.Spring;

public interface Sender {
    void send(String text);
}
```

EmailSender.java

```
package pl.example.Spring;

import org.springframework.stereotype.Component;

@Component
public class EmailSender implements Sender {
    @Override
    public void send(String text) {
        System.out.println("E-mail: " + text);
    }
}
```

SmsSender.java

```
package pl.example.Spring;

import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Component;

@Component
@Primary
public class SmsSender implements Sender {
    @Override
    public void send(String text) {
        System.out.println("SMS: " + text);
    }
}
```

OrderService.java

```
package pl.example.Spring;

import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final Sender sender;

    public OrderService(Sender sender) {
        this.sender = sender;
    }

    public void placeOrder(String product) {
        sender.send("Zamówienie przyjęte: " + product);
    }
}
```

OrderRunner.java (uruchamia się sam po starcie aplikacji)

```
package pl.example.Spring;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class OrderRunner implements CommandLineRunner {

    private final OrderService orderService;

    public OrderRunner(OrderService orderService) {
        this.orderService = orderService;
    }

    @Override
    public void run(String... args) {
        orderService.placeOrder("Książka");
    }
}
```

Po uruchomieniu `Application` w konsoli pojawi się:

```
SMS: Zamówienie przyjęte: Książka
```

Co tu się stało?

1. Spring zeskanował pakiet i znalazł `EmailSender`, `SmsSender`, `OrderService` i `OrderRunner`.
2. `OrderRunner` potrzebuje `OrderService`, a `OrderService` potrzebuje `Sender`.
3. Są dwa `Sender`-y, więc dzięki `@Primary` Spring wybrał `SmsSender`.
4. Spring utworzył obiekty od końca: `SmsSender`, potem `OrderService`, potem `OrderRunner`.
5. Po starcie wywołał `run(...)` w `OrderRunner`.

Nigdzie w kodzie nie ma `new` dla `OrderService`, `Sender` ani `OrderRunner`.